# 西湖大学 AIStation 开发环境及必要组件持久化部署指南

Clash for Linux、VS Code Remote SSH 与 Codex

技术实践记录与配置整理

2026 年 10 月 25 日

**摘要**

本文整理在西湖大学 AIStation GPU 容器环境中持久化配置 Clash、VS Code Server 和 Codex 的方法。目标是避免在容器关闭、重建或更换 GPU 环境时反复下载软件、配置代理、登录 Codex，并尽量保留历史聊天记录。

## 目录

**1 基本原理与部署前提** **3**

**2 第一部分：Clash for Linux 持久化** **3**

2.1 安装目录和文件结构 . . . . . 3

2.2 订阅白名单与出口 IP . . . . . 3

2.3 初始化脚本 . . . . . 3

2.3.1 代理环境变量：clash_env.sh . . . . . 4

2.3.2 环境初始化：init.sh . . . . . 4

2.4 容器重建后的使用方式 . . . . . 4

**3 第二部分：VS Code Remote SSH 持久化** **5**

3.1 Mac 本地 SSH 配置 . . . . . 5

3.2 指定远程 Server 的持久化安装目录 . . . . . 6

3.3 代理与扩展配置 . . . . . 6

**4 第三部分：Codex 登录与聊天记录持久化** **7**

4.1 问题背景 . . . . . 7

4.2 持久化的基本原理 . . . . . 7

4.3 通过 CLI 包装器固定 CODEX_HOME . . . . . 8

4.4 在 VS Code 中配置 Codex 启动路径 . . . . . 8

4.5 通过 Prompt 自动完成持久化配置 . . . . . 9
4.5.1 Codex 自动配置 Prompt . . . . . 9
4.6 Codex 持久化诊断 . . . . . 11

**5 日常使用流程及注意事项 12**

**A Mac 本地配置文件 13**
A.1 SSH 配置文件 . . . . . 13
A.2 VS Code settings.json . . . . . 13

**B 远程环境初始化脚本 14**
B.1 clash_env.sh . . . . . 14
B.2 init.sh . . . . . 15

**C 远程 bin 目录中的 Clash 脚本 15**
C.1 clashctl . . . . . 16
C.2 clashctl-bin . . . . . 16
C.3 clashon . . . . . 16
C.4 clashoff . . . . . 16

**D 远程 bin 目录中的 Codex 脚本 16**
D.1 codex . . . . . 16
D.2 codex-vscode-persistent . . . . . 17
D.3 codex-persistence-doctor . . . . . 18
D.4 codex-persistence-setup . . . . . 20

**E 其他文件与最终命令速查 22**
E.1 日常命令速查 . . . . . 22
E.2 最终说明 . . . . . 23

# 1 基本原理与部署前提

AIStation 采用容器化开发环境。当前实例的 `/root` 位于容器内 overlay 文件系统，容器销毁重建后丢失内容；`/yaochangming` 挂载于 GPFS 持久化存储，可保留程序与用户数据。其他阅读者应将文中的 `/yaochangming` 替换为本人对应的持久化路径。

| 工具 | 持久化对象 | 实际使用目录 |
|---|---|---|
| Clash | 核心程序、订阅、配置 | `/yaochangming/tools/clash-for-linux` |
| VS Code | Server、扩展、数据 | `/yaochangming/vscode-server/ycm_0714` |
| Codex | 登录状态、会话、数据库 | `/yaochangming/.codex` |

进入远程容器后，先确认挂载：

```
findmnt -T /yaochangming
df -h /yaochangming
```

# 2 第一部分：Clash for Linux 持久化

## 2.1 安装目录和文件结构

所用项目可参考：https://github.com/nelvko/clash-for-linux-install。首次安装时应查阅对应版本的 README，将安装目录放在持久化存储，不要直接安装到 `/root`。

## 2.2 订阅白名单与出口 IP

Clash for Linux 仅负责提供本地代理客户端及其运行管理功能，本身不提供代理节点或网络订阅服务。因此，使用前还需要自行选择合适的代理服务商，获取相应的订阅链接，并将其导入 Clash。部分订阅服务商仅提供服务给登陆过该服务商网站的设备，但无图形化的界面的远程服务器很难做到这点。因此可以将服务器公网出口 IP 加入白名单。该 IP 不等于 SSH 登录使用的内网地址。具体查询方式可通过远程终端查询：

```
curl -4 --noproxy '*' https://icanhazip.com
```

## 2.3 初始化脚本

为了避免每次创建新容器时重复配置环境变量，在持久化目录 `/yaochangming/env/` 下创建两个 Shell 脚本：`init.sh` 和 `clash_env.sh`。

其中，`clash_env.sh` 负责设置代理环境变量，`init.sh` 负责初始化持久化命令目录、Codex 数据目录，并加载代理配置。

#### 2.3.1 代理环境变量：clash_env.sh

文件位置：/yaochangming/env/clash_env.sh

当前实际配置如下：

```
export http_proxy=http://127.0.0.1:7890
export https_proxy=http://127.0.0.1:7890
export HTTP_PROXY=http://127.0.0.1:7890
export HTTPS_PROXY=http://127.0.0.1:7890
export ALL_PROXY=socks5://127.0.0.1:7891
```

其中，7890 用于 HTTP/HTTPS 代理，7891 被配置为 SOCKS 代理端口。需要注意，以上为当前实际设置，但是否存在独立的 7891 监听仍需检查。

#### 2.3.2 环境初始化：init.sh

文件位置：/yaochangming/env/init.sh

当前实际配置如下：

```
#!/bin/bash

# Persistent commands
export PATH=/yaochangming/bin:$PATH
export CODEX_HOME=/yaochangming/.codex

# Proxy environment
if [ -f /yaochangming/env/clash_env.sh ]; then
    source /yaochangming/env/clash_env.sh
fi

echo "Westlake persistent environment loaded"
```

该脚本依次完成以下操作：

1. 将持久化的 bin 目录加入 PATH，使终端能够直接调用 clashon 等命令。
2. 将 CODEX_HOME 指向持久化目录，避免 Codex 默认使用非持久化的 /root/.codex。
3. 检查代理环境文件是否存在，存在时通过 source 加载其中的环境变量。
4. 输出初始化成功的提示信息。

### 2.4 容器重建后的使用方式

在新建或重建的 GPU 容器中，确认持久化存储已挂载后，执行：

```
source /yaochangming/env/init.sh
clashon
```

第一条命令加载持久化环境及代理变量，第二条命令通过此前配置的包装脚本启动 Mihomo。初始化后，终端将显示：

```
Westlake persistent environment loaded
```

可通过以下命令检查代理进程及监听端口：

```
ps -ef | grep '[m]ihomo'
ss -lntp | grep -E ':(7890|7891|9090)\b'
```

需要时测试代理连通性以及选择节点：

```
curl -I https://www.google.com
clashctl select
```

# 3 第二部分：VS Code Remote SSH 持久化

## 3.1 Mac 本地 SSH 配置

Mac 终端打开 SSH 配置：

```
nano ~/.ssh/config
```

本人当前使用的三个远程连接示例如下（端口会随容器变化）：

```
Host ycm_0714
    HostName 172.16.78.10
    User root
    Port 35193

Host ycm_v100_test
    HostName 172.16.78.10
    User root
    Port 32841

Host ycm_A100
    HostName 172.16.78.10
    User root
```

```
    Port 40964
```

终端可通过别名连接：

```
ssh ycm_A100
```

### 3.2 指定远程 Server 的持久化安装目录

VS Code Remote SSH 默认可能安装到 `/root/.vscode-server`。在 Mac 本地 VS Code 中，按 `Cmd + Shift + P`，选择 `Preferences: Open User Settings (JSON)`，将下列字段合并进已有 JSON 对象：

```
"remote.SSH.serverInstallPath": {
    "ycm_0714": "/yaochangming/vscode-server/ycm_0714",
    "ycm_v100_test": "/yaochangming/vscode-server/ycm_0714",
    "ycm_A100": "/yaochangming/vscode-server/ycm_0714"
}
```

此方案为多个 GPU 容器**轮流使用**一套安装目录。前提是它们共享同一份持久化数据，并且切换环境之前，旧环境使用该目录的 Server、扩展宿主及 Codex 进程已经退出。**关闭 VS Code 窗口不一定代表远程进程已经退出**。若计划同时运行多个远程环境，应考虑为各容器分配独立可写 Server 目录。

### 3.3 代理与扩展配置

Mac 本地 `settings.json` 中还设置过：

```
"extensions.autoUpdate": "off",
"extensions.autoCheckUpdates": "off",
"chatgpt.localeOverride": "zh-CN"
```

历史配置还包含如下代理字段：

```
"remote.SSH.remoteEnv": {
    "HTTP_PROXY": "http://127.0.0.1:7890",
    "HTTPS_PROXY": "http://127.0.0.1:7890",
    "ALL_PROXY": "socks5://127.0.0.1:7891",
    "http_proxy": "http://127.0.0.1:7890",
    "https_proxy": "http://127.0.0.1:7890",
    "all_proxy": "socks5://127.0.0.1:7891"
}
```

这段是**原配置记录**，实际请根据个人服务器环境来做设置。

# 4 第三部分：Codex 登录与聊天记录持久化

## 4.1 问题背景

Codex 的持久化是整个开发环境配置中相对复杂的部分。仅将 VS Code Server 和 Codex 扩展安装到持久化目录，并不能保证 Codex 的登录状态和聊天记录得到保留。

在以 root 用户运行的容器中，Codex 默认可能使用：

```
/root/.codex
```

该目录位于容器的非持久化文件系统上。容器销毁后，其中的认证文件、聊天记录、会话索引和数据库可能随之丢失。

因此，需要将 Codex 的用户数据目录指定为：

```
/yaochangming/.codex
```

主要使用以下环境变量：

```
export CODEX_HOME=/yaochangming/.codex
```

经过此前的配置和实际检查，目前运行中的 Codex 已经正确使用上述持久化目录，历史聊天记录也已成功保留。

## 4.2 持久化的基本原理

Codex 的持久化主要包含两个相互独立的部分：

1. **扩展程序持久化：** 通过 VS Code 的 `remote.SSH.serverInstallPath` 将 Codex 扩展安装在 GPFS 持久化目录中。
2. **用户数据持久化：**通过 `CODEX_HOME` 将认证信息、聊天记录和状态数据库保存到 `/yaochangming/.codex`。

其中，第二项是保证聊天记录不随容器销毁而丢失的关键。

需要注意，在普通终端执行：

```
export CODEX_HOME=/yaochangming/.codex
```

只会影响当前 Shell 及其子进程，不一定能够影响由 VS Code Server 启动的 Codex 扩展。

因此，还需要通过专用的 CLI 启动包装器，确保 Codex 在启动时使用正确的数据目录。

### 4.3 通过 CLI 包装器固定 CODEX_HOME

此前已经在持久化目录中创建启动脚本：

```
/yaochangming/bin/codex-vscode-persistent
```

该脚本的完整内容如下：

```
#!/bin/sh
set -eu

export CODEX_HOME=/yaochangming/.codex

extensions_root=/yaochangming/vscode-server/ycm_0714/.vscode-server/extensions
codex_binary=

for candidate in "$extensions_root"/openai.chatgpt-*/bin/linux-x86_64/codex; do
    if [ -x "$candidate" ]; then
        codex_binary=$candidate
        break
    fi
done

if [ -z "$codex_binary" ]; then
    echo "Codex extension binary was not found under $extensions_root" >&2
    exit 127
fi

exec "$codex_binary" "$@"
```

该脚本首先设置 `CODEX_HOME`，然后在持久化的 VS Code 扩展目录中寻找 Codex CLI，最后通过 `exec` 启动程序并传递原始参数。

通过这种方式，可以避免 Codex 使用默认的 `/root/.codex`。

需要注意：当前脚本采用通配符查找第一个可执行文件，并不保证多个扩展版本共存时一定选择最新版本。后续升级扩展时应重新检查该逻辑。

### 4.4 在 VS Code 中配置 Codex 启动路径

打开 Mac 本地 VS Code 的 `settings.json`，加入以下设置：

```
"chatgpt.cliExecutable": "/yaochangming/bin/codex-vscode-persistent"
```

此前也在远程 VS Code Server 的 Machine 设置中添加了相同字段：

```
/yaochangming/vscode-server/ycm_0714/.vscode-server/data/Machine/settings.json
```

这样，VS Code Codex 扩展便可以通过指定的包装器启动 Codex CLI，并使用持久化的数据目录。

此前还配置过：

```
/yaochangming/vscode-server/ycm_0714/.vscode-server/server-env-setup
```

但不能仅凭该文件存在便认定环境变量已经生效。实际审计中，Codex 进程的 `CODEX_HOME` 已正确指向持久化目录，而其父级扩展宿主没有相同的环境变量。

因此，当前方案主要依赖 CLI 启动包装器。

## 4.5 通过 Prompt 自动完成持久化配置

实际上，上述 Codex 持久化操作不一定需要手动完成。

如果已经能够通过 VS Code Remote SSH 正常使用 Codex，可以直接向服务器上的 Codex 提交配置任务，让其检查当前环境、创建持久化启动器、修改必要设置并进行验证。

相比手动修改多个文件，这种方法能够减少配置错误。

但是，建议先要求 Codex 检查现有环境，在确认目标目录和修改范围后再允许执行配置。首次登录仍可能需要用户自行完成浏览器授权。

下面提供一份可直接复制的任务提示词。

### 4.5.1 Codex 自动配置 Prompt

```
任务：配置当前 AIStation GPU 容器中的
Codex 登录状态与聊天记录持久化。

背景：
当前容器的 /root 是非持久化目录，
/yaochangming 是 GPFS 持久化挂载目录。
我已经通过 VS Code Remote SSH 连接服务器，
希望容器关闭并重建后，Codex 的认证数据、
聊天记录和会话索引仍能够保留。

目标：
1. 将 Codex 用户数据持久化到
   /yaochangming/.codex
2. 确保 VS Code Codex 扩展使用该目录
3. 避免使用 /root/.codex 保存主要状态
4. 尽量保留已有的登录状态和聊天记录
```

5. 创建可靠的检查与诊断方法

请按照以下步骤执行:

第一阶段: 只读检查
- 检查 /yaochangming 是否为持久化挂载
- 检查当前 VS Code Server 安装位置
- 检查 Codex 扩展和 CLI 的实际路径
- 检查 /root/.codex 与持久化目录
- 检查现有 CODEX_HOME 环境变量
- 检查当前进程实际使用的数据目录
- 不要输出 auth.json 中的敏感内容

第二阶段: 提出配置方案
- 优先保留已有的认证和聊天记录
- 采用 CODEX_HOME 指定持久化目录
- 必要时创建专用 Codex CLI 启动包装器
- 配置 chatgpt.cliExecutable
- 避免破坏已有 VS Code 和 Clash 配置
- 明确哪些文件需要修改或创建

第三阶段: 在我确认方案后执行
- 备份即将修改的配置文件
- 创建或修复持久化启动包装器
- 修改必要的远程 VS Code 设置
- 不要删除原有聊天记录和认证信息
- 不要自动注销或重置 Codex 登录
- 不要将认证文件上传到外部服务

第四阶段: 检查结果
- 检查 Codex 进程的 CODEX_HOME
- 确认认证、sessions 和数据库位置
- 检查是否仍依赖 /root/.codex
- 提供容器重建后的复用操作说明

要求:
优先复用已有配置, 不重复安装软件。
不修改 source_data、weights 等科研数据。
如果需要我手动登录或授权, 请明确说明。
先完成只读检查并报告方案,
未经我确认不要进行写入操作。

**自动配置说明：**这里的 Prompt 是提供给服务器上已经能够运行的 Codex。它可以辅助完成文件检查、脚本创建和配置修改，但不能保证首次账号登录完全自动完成。
如果原有 Codex 已有聊天记录，应先备份并检查数据库状态，避免在服务运行期间直接覆盖 SQLite 数据库。
此外，持久化保存认证数据并不代表认证永不过期。

### 4.6 Codex 持久化诊断

目前的持久化目录中保存了如下数据：

```
/yaochangming/.codex/
    auth.json
    config.toml
    sessions/
    archived_sessions/
    session_index.jsonl
    state_5.sqlite
    thread_history_1.sqlite*
    logs_2.sqlite*
    skills/
    plugins/
    attachments/
```

可以执行：

```
ls -lah /yaochangming/.codex
ps -ef | grep '[c]odex'
```

此外，已配置专用诊断脚本：

```
/yaochangming/bin/codex-persistence-doctor
```

关键检查项包括：

```
persistent_auth=present
vscode_launcher_setting=ok
codex_persistence_status=ok
codex_nonpersistent_processes=0
```

其中：

- `persistent_auth=present`：认证文件存在，不代表认证一定有效。

- `vscode_launcher_setting=ok`：远程 Machine 设置包含持久化启动器路径。
- `codex_persistence_status=ok`：被检查到的 Codex app-server 进程均使用持久化目录。
- `codex_nonpersistent_processes=0`：没有发现被该脚本识别的非持久化 app-server 进程。

最终仍需要在 VS Code 界面中确认登录状态及历史会话是否正常恢复。

## 5 日常使用流程及注意事项

完成首次配置后，后续使用 GPU 容器主要包含以下步骤：

1. 启动 AIStation GPU 开发容器，确认持久化目录挂载正常：

```
findmnt -T /yaochangming
```

2. 初始化 Shell 环境并启动 Clash：

```
source /yaochangming/env/init.sh
clashon
```

3. 在 Mac 本地打开 VS Code，使用 Remote SSH 连接对应的 GPU 容器。
4. 打开 Codex 扩展，检查登录状态和聊天记录是否正常。
5. 如有需要，执行持久化诊断：

```
/yaochangming/bin/codex-persistence-doctor
```

6. 切换 GPU 容器前，确保旧环境中使用共享状态目录的相关进程已退出，避免多个容器同时写入相同的运行数据。

| 工具 | 核心配置 | 持久化目的 |
|---|---|---|
| Clash | `init.sh`、`clashon`、持久化 runtime | 保存程序、订阅和节点配置 |
| VS Code | `remote.SSH.serverInstallPath` | 保存 Server 和远程扩展 |
| Codex | `CODEX_HOME`、`chatgpt.cliExecutable` | 保存认证、聊天记录与状态数据库 |

**跨容器复用说明：**本方案主要针对多个 GPU 容器轮流使用的场景。
同一份持久化目录可以保存程序和用户数据，但不能因此认为多个正在运行的容器能够安全地并发写入相同的 VS Code Server 或 Codex SQLite 数据库。
建议同一时间只使用一个活跃的远程开发环境，并在切换前确认旧进程已退出。

# A Mac 本地配置文件

本附录整理此前实际使用的配置和脚本。所有示例均基于本人环境，其他用户需要修改个人目录、SSH 端口和主机别名。

## A.1 SSH 配置文件

Mac 本地路径：

```
~/.ssh/config
```

完整配置：

```
Host ycm_0714
    HostName 172.16.78.10
    User root
    Port 35193

Host ycm_v100_test
    HostName 172.16.78.10
    User root
    Port 32841

Host ycm_A100
    HostName 172.16.78.10
    User root
    Port 40964
```

**作用：**为不同 GPU 容器定义独立的 SSH Host 别名，从而允许 VS Code 直接通过主机别名连接。

上述端口为此前实际使用的端口，创建新容器后应以 AIStation 分配的端口为准。

## A.2 VS Code settings.json

Mac 本地文件位置：

```
~/Library/Application Support/Code/User/settings.json
```

原有配置如下：

```
{
    "trae.chatLanguage": "cn",
```

```
    "chatgpt.localeOverride": "zh-CN",

    "extensions.autoUpdate": "off",
    "extensions.autoCheckUpdates": "off",

    "chatgpt.cliExecutable": "/yaochangming/bin/codex-vscode-persistent",

    "remote.SSH.serverInstallPath": {
        "ycm_0714": "/yaochangming/vscode-server/ycm_0714",
        "ycm_v100_test": "/yaochangming/vscode-server/ycm_0714",
        "ycm_A100": "/yaochangming/vscode-server/ycm_0714"
    },

    "remote.SSH.remoteEnv": {
        "HTTP_PROXY": "http://127.0.0.1:7890",
        "HTTPS_PROXY": "http://127.0.0.1:7890",
        "ALL_PROXY": "socks5://127.0.0.1:7891",
        "http_proxy": "http://127.0.0.1:7890",
        "https_proxy": "http://127.0.0.1:7890",
        "all_proxy": "socks5://127.0.0.1:7891"
    }
}
```

主要作用如下：

- `remote.SSH.serverInstallPath`：将 VS Code Server 安装到持久化目录。
- `chatgpt.cliExecutable`：指定 Codex 使用自定义启动器。
- `extensions.autoUpdate`：禁止扩展自动更新。
- `remote.SSH.remoteEnv`：保存原有代理设置；其实际生效情况仍需根据 Remote SSH 版本和远程进程环境检查。

# B 远程环境初始化脚本

## B.1 clash_env.sh

路径：

```
/yaochangming/env/clash_env.sh
```

完整源码：

```
export http_proxy=http://127.0.0.1:7890
export https_proxy=http://127.0.0.1:7890
export HTTP_PROXY=http://127.0.0.1:7890
export HTTPS_PROXY=http://127.0.0.1:7890
export ALL_PROXY=socks5://127.0.0.1:7891
```

**作用：**配置当前 Shell 中的 HTTP、HTTPS 和 SOCKS 代理变量。

### B.2 init.sh

路径：

```
/yaochangming/env/init.sh
```

完整源码：

```
#!/bin/bash

# Persistent commands
export PATH=/yaochangming/bin:$PATH
export CODEX_HOME=/yaochangming/.codex

# Proxy environment
if [ -f /yaochangming/env/clash_env.sh ]; then
    source /yaochangming/env/clash_env.sh
fi

echo "Westlake persistent environment loaded"
```

**作用：**初始化持久化命令目录、Codex 数据目录和代理环境变量。

## C 远程 bin 目录中的 Clash 脚本

下面的脚本均位于：

```
/yaochangming/bin/
```

### C.1 clashctl

```
#!/usr/bin/env bash
set -euo pipefail
exec "/yaochangming/bin/clashctl-bin" "$@"
```

**作用：**作为 Clash 控制命令的统一入口。

### C.2 clashctl-bin

```
#!/usr/bin/env bash
set -euo pipefail
exec bash "/yaochangming/tools/clash-for-linux/scripts/core/clashctl.sh" "$@"
```

**作用：**调用 Clash for Linux 项目中的实际控制脚本。

### C.3 clashon

```
#!/usr/bin/env bash
set -euo pipefail
export CLASH_WRAPPER_EXEC="1"
source "/yaochangming/tools/clash-for-linux/scripts/core/alias.sh"
clashon "$@"
```

**作用：**加载 Clash 项目的 Shell 函数，然后调用其中的 `clashon`，启动 Mihomo 服务。

### C.4 clashoff

```
#!/usr/bin/env bash
set -euo pipefail
export CLASH_WRAPPER_EXEC="1"
source "/yaochangming/tools/clash-for-linux/scripts/core/alias.sh"
clashoff "$@"
```

**作用：**通过原有 Clash 控制逻辑停止 Mihomo 服务。

## D 远程 bin 目录中的 Codex 脚本

### D.1 codex

该脚本用于在远程终端中启动独立的 Codex CLI。

完整源码：

```
#!/usr/bin/env bash

export CODEX_HOME=/yaochangming/.codex

export HTTP_PROXY=http://127.0.0.1:17890
export HTTPS_PROXY=http://127.0.0.1:17890
export http_proxy=http://127.0.0.1:17890
export https_proxy=http://127.0.0.1:17890

export NO_PROXY=127.0.0.1,localhost
export no_proxy=127.0.0.1,localhost

exec /yaochangming/tools/codex/codex-real "$@"
```

**作用：** 指定终端 Codex CLI 使用持久化数据目录，并使用其独立配置的 HTTP 代理端口 17890。

注意，17890 与 Clash 环境脚本中的 7890、7891 不同，应根据实际监听情况检查。

## D.2 codex-vscode-persistent

该脚本用于 VS Code 中的 Codex 扩展，是聊天记录持久化的重要组成部分。

完整源码：

```
#!/bin/sh
set -eu

export CODEX_HOME=/yaochangming/.codex

extensions_root=/yaochangming/vscode-server/ycm_0714/.vscode-server/extensions
codex_binary=

for candidate in "$extensions_root"/openai.chatgpt-*/bin/linux-x86_64/codex; do
    if [ -x "$candidate" ]; then
        codex_binary=$candidate
        break
    fi
done

if [ -z "$codex_binary" ]; then
    echo "Codex extension binary was not found under $extensions_root" >&2
    exit 127
```

```
fi

exec "$codex_binary" "$@"
```

**作用：**在启动扩展附带的 Codex CLI 前，固定 `CODEX_HOME`，使认证和聊天数据保存在 GPFS。

### D.3 codex-persistence-doctor

该脚本用于检查 Codex 的持久化状态。

完整源码：

```
#!/bin/sh
set -eu

persistent_home=/yaochangming/.codex
settings=/yaochangming/vscode-server/ycm_0714/.vscode-server/data/Machine/settings.」
↪ json

echo "hostname=$(hostname)"
echo "persistent_home=$persistent_home"
echo "persistent_home_type=$(stat -c %F "$persistent_home" 2>/dev/null || echo
↪ missing)"
echo "persistent_auth=$(test -f "$persistent_home/auth.json" && echo present || echo
↪ missing)"
echo "persistent_session_files=$(find "$persistent_home/sessions" -type f
↪ 2>/dev/null | wc -l)"
echo "vscode_settings=$settings"

if [ -f "$settings" ] && grep -q '/yaochangming/bin/codex-vscode-persistent'
↪ "$settings"; then
    echo "vscode_launcher_setting=ok"
else
    echo "vscode_launcher_setting=missing"
fi

codex_count=0
persistent_count=0
nonpersistent_count=0

for pid in $(pgrep -x codex 2>/dev/null || true); do
    [ -r "/proc/$pid/cmdline" ] || continue
    cmdline=$(tr '\0' ' ' < "/proc/$pid/cmdline")
    case "$cmdline" in
```

```
        *app-server*) ;;
        *) continue ;;
    esac

    codex_count=$((codex_count + 1))
    codex_home=$(tr '\0' '\n' < "/proc/$pid/environ" | sed -n 's/^CODEX_HOME=//p')
    parent_pid=$(ps -o ppid= -p "$pid" | tr -d ' ')
    started=$(ps -o lstart= -p "$pid" | sed 's/^ *//')

    echo "codex[$pid].parent=$parent_pid"
    echo "codex[$pid].started=$started"
    echo "codex[$pid].home=${codex_home:-unset}"

    if [ "$codex_home" = "$persistent_home" ]; then
        persistent_count=$((persistent_count + 1))
        echo "codex[$pid].persistence=ok"
    else
        nonpersistent_count=$((nonpersistent_count + 1))
        echo "codex[$pid].persistence=wrong"
    fi
done

echo "codex_processes=$codex_count"
echo "codex_persistent_processes=$persistent_count"
echo "codex_nonpersistent_processes=$nonpersistent_count"

if [ "$codex_count" -eq 0 ]; then
    echo "codex_persistence_status=not-running"
elif [ "$nonpersistent_count" -eq 0 ]; then
    echo "codex_persistence_status=ok"
elif [ "$persistent_count" -eq 0 ]; then
    echo "codex_persistence_status=wrong"
else
    echo "codex_persistence_status=mixed"
fi

if [ -L /root/.codex ]; then
    echo "root_codex=$(readlink -f /root/.codex)"
elif [ -d /root/.codex ]; then
    echo "root_codex=container-local-directory"
else
    echo "root_codex=missing"
fi
```

**作用：**检查认证文件、会话数量、远程启动器设置，以及运行中的 Codex app-server 进程是

否使用持久化目录。

该脚本主要进行状态读取，不负责安装、修复或修改配置。

### D.4 codex-persistence-setup

该脚本用于自动配置远程 VS Code Server，将 Codex CLI 启动路径写入 Machine 设置。

完整源码：

```
#!/usr/bin/env python3
"""Install the persistent Codex launcher into a persistent VS Code Server."""

from __future__ import annotations

import argparse
import json
import os
from pathlib import Path
import shutil
import tempfile
from datetime import datetime


DEFAULT_SERVER_ROOT = Path(
    "/yaochangming/vscode-server/ycm_0714/.vscode-server"
)
LAUNCHER = Path("/yaochangming/bin/codex-vscode-persistent")
CODEX_HOME = Path("/yaochangming/.codex")


def main() -> int:
    parser = argparse.ArgumentParser(
        description="Configure a VS Code Server to launch Codex with persistent
        ↪  state."
    )
    parser.add_argument(
        "--vscode-server-root",
        type=Path,
        default=DEFAULT_SERVER_ROOT,
        help="Path containing the VS Code Server data and extensions directories.",
    )
    args = parser.parse_args()

    server_root = args.vscode_server_root.resolve()
    machine_dir = server_root / "data" / "Machine"
```

```
    settings_path = machine_dir / "settings.json"

    if not CODEX_HOME.is_dir():
        raise SystemExit(f"Persistent Codex home does not exist: {CODEX_HOME}")
    if not LAUNCHER.is_file():
        raise SystemExit(f"Persistent launcher does not exist: {LAUNCHER}")
    if not server_root.is_dir():
        raise SystemExit(f"VS Code Server root does not exist: {server_root}")

    machine_dir.mkdir(parents=True, exist_ok=True)
    settings: dict[str, object] = {}
    if settings_path.exists():
        settings = json.loads(settings_path.read_text(encoding="utf-8"))
        stamp = datetime.now().strftime("%Y%m%d-%H%M%S")
        backup = settings_path.with_name(f"settings.json.before-codex-{stamp}")
        shutil.copy2(settings_path, backup)
        print(f"Backed up settings: {backup}")

    settings["chatgpt.cliExecutable"] = str(LAUNCHER)

    fd, temporary_name = tempfile.mkstemp(
        prefix="settings.json.", dir=machine_dir, text=True
    )
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as handle:
            json.dump(settings, handle, ensure_ascii=False, indent=4)
            handle.write("\n")
        os.chmod(temporary_name, 0o644)
        os.replace(temporary_name, settings_path)
    except BaseException:
        try:
            os.unlink(temporary_name)
        except FileNotFoundError:
            pass
        raise

    print(f"Configured settings: {settings_path}")
    print(f"Codex launcher: {LAUNCHER}")
    print(f"Persistent CODEX_HOME: {CODEX_HOME}")
    print("Reload or reconnect the VS Code Remote-SSH window to apply the setting.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

**作用：** 读取远程 VS Code Machine 设置文件，先备份已有配置，再写入持久化 Codex 启动器路径。

**注意：** `codex-persistence-setup` 会实际修改设置文件，不是只读诊断工具，也不是每次启动环境都需要运行的命令。
已有配置正常时，无需重复执行。

# E 其他文件与最终命令速查

## E.1 日常命令速查

```
# Check persistent storage
findmnt -T /yaochangming

# Initialize environment
source /yaochangming/env/init.sh

# Start Clash
clashon

# Select proxy node
clashctl select

# Stop Clash
clashoff

# Check Mihomo process
ps -ef | grep '[m]ihomo'

# Check proxy ports
ss -lntp | grep -E ':(7890|7891|17890)\b'

# Connect from Mac
ssh ycm_A100

# Check VS Code Server
ps -ef | grep '[s]erver-main.js'

# Check Codex files
ls -lah /yaochangming/.codex

# Check Codex process
ps -ef | grep '[c]odex'
```

```
# Diagnose persistence
/yaochangming/bin/codex-persistence-doctor
```

## E.2 最终说明

本方案通过 GPFS 持久化存储，将代理工具、远程开发程序与 Codex 会话数据从容器的临时文件系统中分离出来。

其中：

- Clash 主要解决程序、订阅与运行配置的复用。
- VS Code Remote SSH 主要解决 Server 与扩展的复用。
- Codex 主要解决认证状态、历史会话和数据库的保留。

对于 Codex 等配置较复杂的工具，可以借助已经能够运行的 AI 编程助手，通过明确的任务提示词完成环境审计、配置和验证，从而减少手工修改多个文件的工作量。

但仍应遵循先检查、再备份、最后修改的顺序，同时避免不同容器并发写入同一套持久化运行数据。